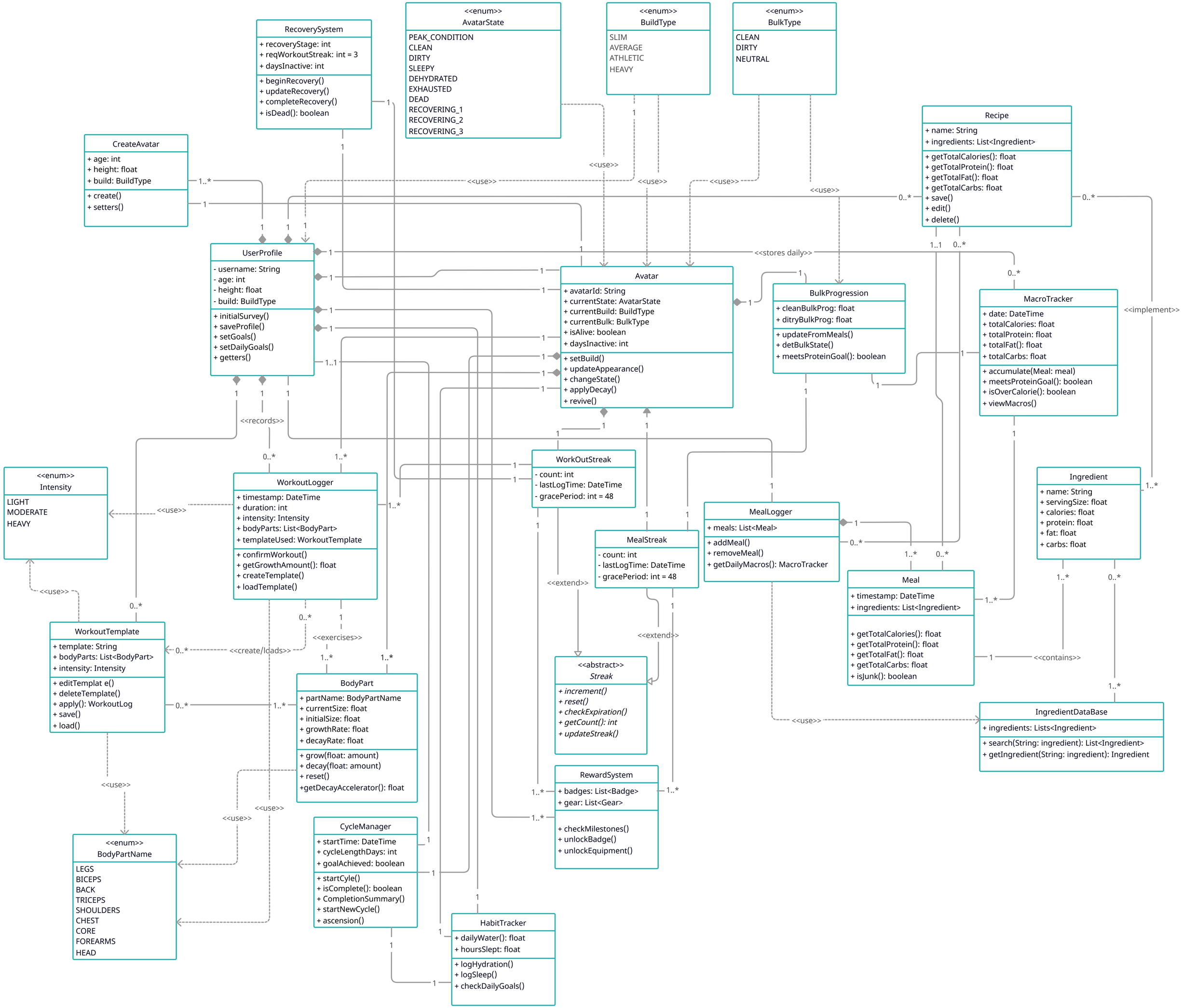

RecoverySystem
+ recoveryStage: int
+ reqWorkoutStreak: int = 3
+ daysInactive: int
+ beginRecovery()
+ updateRecovery()
+ completeRecovery()
+ isDead(): boolean
<<enum>>
AvatarState
PEAK_CONDITION
CLEAN
DIRTY
SLEEPY
DEHYDRATED
EXHAUSTED
DEAD
RECOVERING_1
RECOVERING_2
RECOVERING_3
<<enum>>
BuildType
SLIM
AVERAGE
ATHLETIC
HEAVY
<<enum>>
BulkType
CLEAN
DIRTY
NEUTRAL
CreateAvatar
+ age: int
+ height: float
+ build: BuildType
+ create()
+ setters()
Recipe
+ name: String
+ ingredients: List<Ingredient>
+ getTotalCalories(): float
+ getTotalProtein(): float
+ getTotalFat(): float
+ getTotalCarbs: float
+ save()
+ edit()
+ delete()
UserProfile
- username: String
- age: int
- height: float
- build: BuildType
+ initialSurvey()
+ saveProfile()
+ setGoals()
+ setDailyGoals()
+ getters()
Avatar
+ avatarId: String
+ currentState: AvatarState
+ currentBuild: BuildType
+ currentBulk: BulkType
+ isAlive: boolean
+ daysInactive: int
+ setBuild()
+ updateAppearance()
+ changeState()
+ applyDecay()
+ revive()
BulkProgression
+ cleanBulkProg: float
+ ditryBulkProg: float
+ updateFromMeals()
+ detBulkState()
+ meetsProteinGoal(): boolean
MacroTracker
+ date: DateTime
+ totalCalories: float
+ totalProtein: float
+ totalFat(): float
+ totalCarbs: float
+ accumulate(Meal: meal)
+ meetsProteinGoal(): boolean
+ isOverCalorie(): boolean
+ viewMacros()
<<use>>
<<stores daily>>
<<implement>>
<<records>>
<<enum>>
Intensity
LIGHT
MODERATE
HEAVY
WorkoutLogger
+ timestamp: DateTime
+ duration: int
+ intensity: Intensity
+ bodyParts: List<BodyPart>
+ templateUsed: WorkoutTemplate
+ confirmWorkout()
+ getGrowthAmount(): float
+ createTemplate()
+ loadTemplate()
WorkOutStreak
- count: int
- lastLogTime: DateTime
- gracePeriod: int = 48
MealStreak
- count: int
- lastLogTime: DateTime
- gracePeriod: int = 48
MealLogger
+ meals: List<Meal>
+ addMeal()
+ removeMeal()
+ getDailyMacros(): MacroTracker
Ingredient
+ name: String
+ servingSize: float
+ calories: float
+ protein: float
+ fat: float
+ carbs: float
Meal
+ timestamp: DateTime
+ ingredients: List<Ingredient>
+ getTotalCalories(): float
+ getTotalProtein(): float
+ getTotalFat(): float
+ getTotalCarbs: float
+ isJunk(): boolean
<<contains>>
<<extend>>
WorkoutTemplate
+ template: String
+ bodyParts: List<BodyPart>
+ intensity: Intensity
+ editTemplat e()
+ deleteTemplate()
+ apply(): WorkoutLog
+ save()
+ load()
<<create/loads>>
<<exercises>>
BodyPart
+ partName: BodyPartName
+ currentSize: float
+ initialSize: float
+ growthRate: float
+ decayRate: float
+ grow(float: amount)
+ decay(float: amount)
+ reset()
+getDecayAccelerator(): float
<<abstract>>
Streak
+ increment()
+ reset()
+ checkExpiration()
+ getCount(): int
+ updateStreak()
IngredientDataBase
+ ingredients: Lists<Ingredient>
+ search(String: ingredient): List<Ingredient>
+ getIngredient(String: ingredient): Ingredient
<<enum>>
BodyPartName
LEGS
BICEPS
BACK
TRICEPS
SHOULDERS
CHEST
CORE
FOREARMS
HEAD
CycleManager
+ startTime: DateTime
+ cycleLengthDays: int
+ goalAchieved: boolean
+ startCyle()
+ isComplete(): boolean
+ CompletionSummary()
+ startNewCycle()
+ ascension()
RewardSystem
+ badges: List<Badge>
+ gear: List<Gear>
+ checkMilestones()
+ unlockBadge()
+ unlockEquipment()
HabitTracker
+ dailyWater(): float
+ hoursSlept: float
+ logHydration()
+ logSleep()
+ checkDailyGoals()